

CrossBit: Bitwise Computing in NAND Flash Memory with Inter-Bitline Data Communication

Hyunjin Kim
Seoul National University
Seoul, Republic of Korea
hyunjin.kim@snu.ac.kr

Seunghwan Song
Samsung Electronics
Hwaseong, Republic of Korea
sh27.song@samsung.com

Sukhyun Choi
Seoul National University
Seoul, Republic of Korea
sukhyunchoi@snu.ac.kr

Jeongin Choe
Samsung Electronics
Hwaseong, Republic of Korea
ji.choe@samsung.com

Sanghyeok Han
Seoul National University
Seoul, Republic of Korea
hansh778@snu.ac.kr

Jisung Park
POSTECH (Pohang University of
Science and Technology)
Pohang, Republic of Korea
jisung.park@postech.ac.kr

Jinho Lee
Seoul National University
Seoul, Republic of Korea
leejinho@snu.ac.kr

Jae-Joon Kim
Seoul National University
Seoul, Republic of Korea
kimjaejuon@snu.ac.kr

Abstract

In-flash processing (IFP), which involves performing data computation inside NAND flash memory, holds high potential for improving the performance and energy efficiency of data-intensive application by minimizing data movement. Recent research has introduced several IFP architectures enabling bulk bitwise operations inside NAND flash chips to demonstrate this potential. However, previous IFP designs were limited to performing bitwise operations on data sensed within the same bitline, thus lacking the capability to handle more complex functions requiring interactions between data from different bitlines. This paper presents CrossBit, a new IFP architecture that enables both *intra-bitline* and *inter-bitline operations* with minimal additional circuitry integrated into commodity NAND flash memory. With the capability for *inter-bitline operations*, CrossBit facilitates in-flash error correction code (IF-ECC) operations, thereby enabling reliable multi-level cell (MLC) IFP. Moreover, CrossBit efficiently processes fundamental database queries that were previously inefficient with existing work that only supports *intra-bitline operations*. Experimental results show that the implementation of IF-ECC in CrossBit results in a substantial reduction in bit error rate (BER) for MLC operations, leading to 1.8 $\times$ increase in bit-density by using MLC compared to previous IFP designs which uses SLC only. When used for accelerating fundamental database queries, CrossBit achieves notable average speedup and energy efficiency improvement of 2.1 $\times$ and 2.5 $\times$ compared to the state-of-the-art (SOTA) IFP architecture. We further demonstrate the practicality by processing the full set of end-to-end database queries from the widely used Star schema benchmark, where CrossBit achieves 1.7 $\times$ speedup.



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License.

MICRO '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1573-0/25/10

<https://doi.org/10.1145/3725843.3756055>

Keywords

PIM, NAND flash memory, bulk bitwise operation, dynamic circuit

ACM Reference Format:

Hyunjin Kim, Seunghwan Song, Sukhyun Choi, Jeongin Choe, Sanghyeok Han, Jisung Park, Jinho Lee, and Jae-Joon Kim. 2025. CrossBit: Bitwise Computing in NAND Flash Memory with Inter-Bitline Data Communication. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3725843.3756055>

1 Introduction

In-flash processing (IFP), which supports computation inside NAND flash chips, has demonstrated its high potential to improve the performance and energy efficiency of data-intensive applications by fundamentally mitigating data movements between compute units (e.g., CPUs and GPUs) and memory hierarchy [13, 24, 60]. IFP offers two key advantages compared to other near-data processing (NDP) approaches at higher levels of the memory hierarchy, e.g., CPU cache [2, 20] and main memory [5, 25, 55]. First, for various data-intensive applications where the data size often cannot fit into the main memory, IFP offers ample capacity at a lower cost [18, 54] due to the high bit density of stacked NAND flash memory [36]. Second, IFP minimizes data movements through the memory hierarchy, as NAND flash memory typically serves as the lowest-level storage medium, maximizing NDP's performance and energy benefits.

Despite IFP's high potential, we identify two key limitations of existing IFP architectures [24, 60]. First, existing IFP architectures have significant restrictions in *inter-bitline computations*, i.e., computation on bits that are sensed through different bitlines (BLs). Specifically, ParaBit [24] and Flash-Cosmos [60] can perform bitwise operations only when the target bits are sensed through the same BL. AresFlash [13] partially addresses the limitation by supporting shift operation with additional circuitry at the page buffer, but it allows only unidirectional data transfer (e.g., lower- to higher-index BLs) within small ranges of associated BLs (up to 32 BLs). Such limited flexibility in inter-BL computation, in turn, mandates

either cell programming of intermediate results (in ParaBit and Flash-Cosmos) or additional data movements within the page buffer (in AresFlash) to support complex logic computations, diminishing their benefits significantly. Second, the existing IFP techniques *cannot* support reliable computation in multi-level cell (MLC) NAND flash memory due to the lack of error-correction capability inside NAND flash chips. Even though prior work has experimentally demonstrated the possibility of reliable IFP in single-level cell (SLC) flash memory at the cost of program latency [60], it significantly limits the key advantage of IFP over other NDP approaches, i.e., the high storage capacity.

To address the limitations, we propose *CrossBit*, a new IFP architecture that enables (1) *flexible inter-BL computation* and (2) *efficient in-flash error correction*. First, we introduce a new design of page buffer that enables CrossBit to perform inter-BL computation on arbitrary bits within a large chunk of the page buffer. We achieve this by adding only minimal circuitry to the conventional page buffer, ensuring compatibility with conventional memory operations. CrossBit architecture effectively implements thousands of processing elements per plane, thereby providing extensive parallelism across hundreds of NAND flash planes. With our programming method specially optimized for CrossBit, applications can efficiently perform various logic operations as it is Boolean-complete. Second, we develop a new in-flash ECC mechanism, called *In-Flash ECC* (IF-ECC), which leverages CrossBit's inter-BL computation to support the Hamming code [40, 63] inside NAND flash chips without additional hardware logic. With a slight reduction in the coding rate (i.e., more ECC-parity bits), our IF-ECC meets the reliability requirements specified in the JEDEC standard [1], effectively achieving 1.8× higher bit density compared to the existing IFP architectures.

To evaluate CrossBit, we use the state-of-the-art SSD simulator (MQSim) [68], integrated with HSPICE [67] circuit simulation to model the additional inter-BL computing unit. We compare CrossBit with other IFP architectures [13, 24, 60], in-storage processing that performs computation in the SSD controller [26, 27], and conventional host-side CPU processing. In our experiments conducted over the entire Star Schema Benchmark [59] queries that involve the fundamental database queries and other logic operations, such as addition and multiplication, CrossBit achieves 1.7× speedup over the state-of-the-art architecture, while incurring only 2.2% area overhead in NAND flash die. It is also worth noting that CrossBit is compatible with Open NAND Flash Interface (ONFI), a standard NAND flash memory interface, allowing seamless integration with existing NAND flash memory systems and interfaces.

Our key contributions can be summarized as follows:

- We propose CrossBit architecture with *inter-bitline operation* feature within NAND flash memory, providing complex computation capabilities.
- To implement efficient and low-overhead inter-bitline operations, we devise dynamic logic-based computing modules and a hierarchical architecture of the modules, which embraces existing flash circuit design.
- We propose a programming method for CrossBit with optimization methods to reduce computational overhead.
- Using CrossBit, we propose the in-flash error detection/correction method for the use of MLC in IFP.

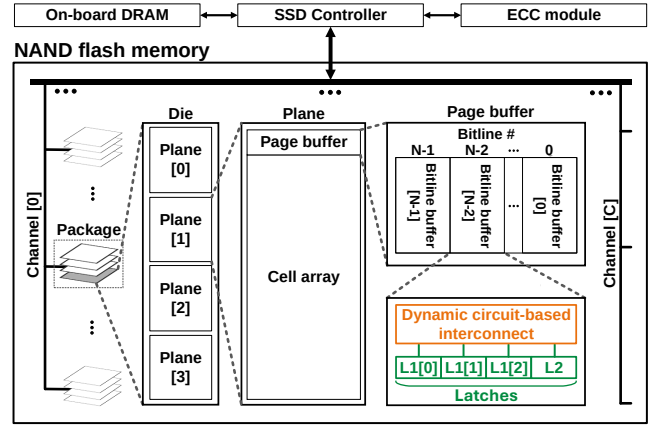


Figure 1: Modern SSD architecture.

- We demonstrate the feasibility of CrossBit with the conventional interface protocol, and achieve a significant speedup on a full set of end-to-end SSB queries.

2 Background and Related Work

2.1 SSD Architecture

SSD comprises NAND flash memory that stores data and SSD controller that manages its operations (Figure 1(a)). NAND flash memory employs a parallel operational structure, organized into multiple channels, packages, dies, and planes. A plane consists of cell array, page buffer, and peripheral circuits. Each row of a cell array is called page (e.g., 16KiB). Cell read and write occur at page granularity, using page buffer as a temporary storage. Page buffer consists of bitline buffers paired with each bitline.

As page buffers occupy a large portion of peripheral circuit area, its design is highly optimized for area efficiency using dynamic circuits [34]. As shown in Figure 1 (a), each bitline buffer includes multiple latches (L1[0:2] and L2), and a dynamic circuit-based interconnect (DC-interconnect) for data communication between latches. DC-interconnect consumes significantly low area by using dynamic circuits instead of conventional static circuits. Dynamic circuits represent a digital value of 1 by holding charge on an electrically isolated wire node, rather than continuously driving the value 1 with transistors as in static circuits [6, 19, 47, 73]. With dynamic circuits, DC-interconnect of a bitline buffer reduces area overhead by 5× compared to the design with static circuits. Section 4.2 details DC-interconnect operation.

2.2 Previous In-Flash Processing Architectures

IFP architecture computes inside NAND flash, thereby reducing the amount of data transfer between NAND flash and external devices. By leveraging internal read bandwidth between cell array and page buffer, along with plane-level parallelism, IFP achieves throughput significantly exceeding external I/O bandwidth. ParaBit [24] proposed the first IFP that performs bulk bitwise logic operations to the best of our knowledge. For logic operation, it applies appropriate read voltages to memory cells and its variation ParaBit-LocFree conducts *intra-bitline operations* in page buffer. Flash-Cosmos [60], on

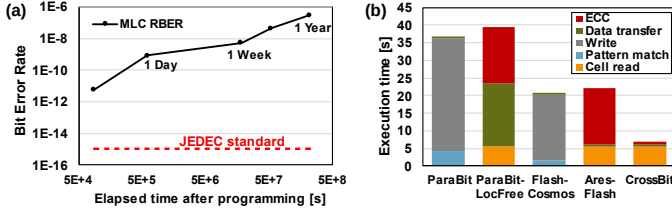


Figure 2: (a) Raw bit error rate (RBER) of MLC with enhanced programming method of Flash-Cosmos and (b) execution time of various IFPs for a pattern match query.

the other hand, proposed an IFP for logic operations that leverages the internal parallel connection of memory cells inside cell array to enhance parallelism compared to ParaBit. Ares-Flash [13] proposed an IFP that supports shift operation for addition and multiplication. It adds additional circuit to page buffer that enables unidirectional data transfer between adjacent BLs.

2.3 In-Storage Processing

In-storage processing (ISP) integrates additional computing unit within SSD controller to reduce data transfer from SSD controller to host [26, 27, 38, 48, 76]. While ISP provides more advanced operations than IFP, the throughput is limited by channel bandwidth as the data has to be serially transferred through channel from NAND flash to SSD controller. In contrast, IFP achieves throughput surpassing the channel bandwidth by computing before data transfer through channel [60]. [76] proposed an inter-channel computation for ISP, sharing a similar goal with CrossBit in terms of enabling computation between physically separated data. However, it still suffers from channel bandwidth bottleneck, resulting in lower performance than IFP-based CrossBit.

3 Challenges of Existing IFP Architectures

3.1 Reliability Issue of MLC

One of the key challenges of prior IFP architectures is their inability to perform ECC operations, restricting them to use only SLC. IFP architectures cannot utilize the conventional ECC module in SSD controller as computations occur before the data transfer to the ECC module at the SSD controller, i.e., outside NAND flash chips. Without ECC, achieving a sufficiently low BER is infeasible, even with error-robust SLCs [60]. To address this issue, Flash-Cosmos [60] proposes a special cell programming scheme that enhances the margin between adjacent V_{TH} levels.¹ However, Flash-Cosmos reports that its BER of MLC cannot satisfy the JEDEC standard, which our own simulation also confirms (Figure 2 (a)). Despite the challenges, pressure to use MLC in IFPs is immense, especially for data-intensive applications, as insufficient capacity would cause additional overhead in moving data between memory devices. One naive alternative could be to place a separate ECC module inside NAND flash dies [15, 50]. However, such a design mandates serial data streaming into the ECC module, losing its internal parallelism.

¹A flash cell stores data using its V_{TH} level that is dictated by the amount of charge in the cell. The larger the margin between adjacent V_{TH} levels (which represent different bit data), the higher the reliability of stored data.

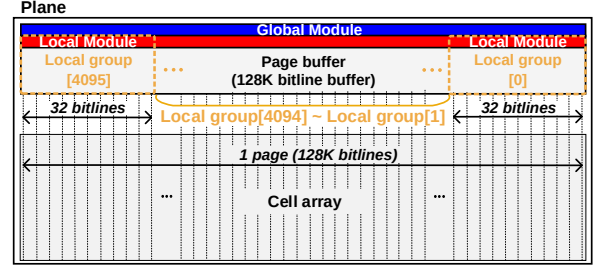


Figure 3: Overall architecture of CrossBit. Blue (global module) and red (local module) boxes are newly added modules.

3.2 Lack of Generic Inter-Bitline Operations

Previous IFPs such as ParaBit [24] and Flash-Cosmos [60], incur significant cell write overhead to compute complex logic. They operate on data at specific physical addresses. Therefore, complex logic operations involving external inputs, intermediate results, or randomly given workload lead to frequent write. It degrades performance as cell programming latency is an order of magnitude higher than cell read latency in NAND flash [14, 33]. For example, ParaBit and Flash-Cosmos spend 87.0% and 84.8%, respectively, of their execution time on write for pattern match as shown in Figure 2(b). Additionally, as they support intra-bitline computations only, conventional row-wise data must be transposed. It requires multiple row accesses to read entire bits during conventional read. Furthermore, these architectures cannot support ECC, even with an ECC module, because they must reprogram error-corrected data into flash cells, introducing new errors. Therefore, we do not include ECC overhead for them in Figure 2(b).

ParaBit-LocFree [24], another existing IFP, avoids cell write by utilizing intra-bitline computation of page buffer, but it supports only limited operations. To process pattern match query, which checks if stored data matches input data, it separately evaluates each bit positions stored in individual bitlines. Since it cannot compute across bitlines, match results for each bit position must be transferred externally to determine full matches, causing substantial data movement as shown in Figure 2(b). In the figure, we include ECC overhead for ParaBit-LocFree, supposing separate ECC module.

Ares-Flash [13] connects neighboring bitline buffers for shift operations. It improves computational flexibility of page buffer and reduces data transfers. However, it provides only unidirectional data transfers between adjacent bitlines, limiting computations and increasing data movement within page buffer. Therefore, ECC computation for MLC is more efficient with a separate ECC module in Ares-Flash (Section 8.2). Figure 2(b) shows that even with the ECC module, ECC takes up 72% of execution time of Ares-Flash. CrossBit, the proposed IFP with inter-bitline operations in page buffer, can address the problem. It provides complex logic computation capability, while avoiding cell write overhead. As shown in Figure 2(b), CrossBit efficiently processes computations of ECC, reducing overall execution time.

4 CrossBit Architecture

4.1 Overview

To address the aforementioned challenges, we propose a novel architecture to perform inter-bitline operations in an unchanged data layout (i.e., words are spread across multiple bitlines). This allows for flexible computations, enabling various complex computations including in-IFP ECC capability to achieve low BER (to be discussed in 6.3). In order to design efficient and practical inter-bitline computing modules, we established the following design objectives:

- (1) High area efficiency should be ensured to keep the bit density of NAND flash memory superior.
- (2) The bitline-level parallelism should be exploited to achieve high performance.
- (3) Boolean-completeness should be guaranteed to ensure versatility across various applications.

To meet these objectives, CrossBit employs two approaches: designing dynamic circuit-based Boolean-complete computing units in page buffer and implementing a highly parallel architecture adopting hierarchical structure.

As depicted in Figure 3, CrossBit introduces inter-bitline operations within each plane. CrossBit is organized into multiple *local groups*, each comprising predefined number of bitline buffers and a local inter-bitline computing module (*local module*). Initially, within a *local group*, *local module* processes data stored in different bitlines buffers. Global inter-bitline computing module (*global module*) further computes between the results obtained from *local modules*.

4.2 Computing Units

In CrossBit, we provide an efficient design that embraces the page buffers already existing in modern NAND flash peripherals. To achieve both efficiency and parallelism, we design Crossbit computing units in three levels of hierarchy using computing latch structure as shown in Figure 4 and Figure 5. First, we show that the existing modern page buffer can be used for intra-bitline operations just by using a set of control signals without any additional hardware (Section 4.2.1). Second, we utilize the same structure to build local inter-bitline modules with a minimal overhead (Section 4.2.2). Last, we apply dynamic circuit technique utilized in page buffer to build low-cost circuits for global operations (Section 4.2.3).

4.2.1 Intra-Bitline Computing Module. Figure 4 (a) illustrates the structure of a bitline buffer which is used in modern flash memories [12, 14]. It consists of latches and dynamic circuit-based interconnect (DC-interconnect). Its original purpose is just to buffer the data in latches using Set and Reset, where Shared_node is used to transfer the data between the latches. Using this structure, we define a few *intra-bitline* computational operations as follows:

- **init** (Figure 4 (b)): Charge=0 charges Shared_node to 1.
- **d-NOR** (Figure 4 (c)): Select=1 discharges Shared_node to 0 when Q=1. Performing this across independent Select signals of multiple latches reflects the logical NOR result between Q values of the selected latches (Select=1) onto Shared_node (i.e., it discharges Shared_node to 0 if at least one Q of selected latches is 1).
- **store** (Figure 4 (d)): When Shared_node=1, Set=1/Reset=1 overwrite the Q value to 1 (case 1.1 in the figure)/0 (case 2.1 in the figure), respectively. Otherwise, when Shared_node=0, it disconnects the

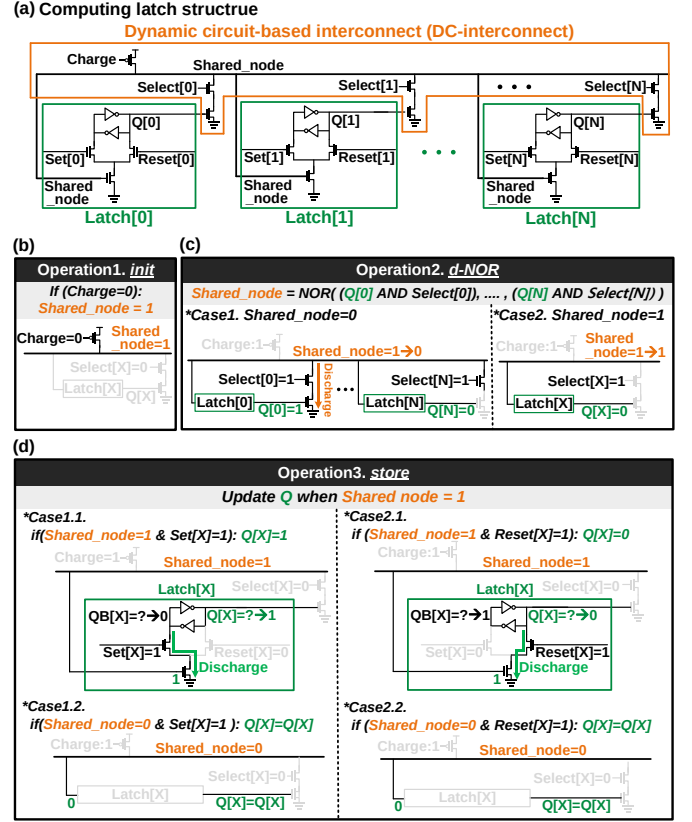


Figure 4: (a) Dynamic logic-based computing latch, and three basic operations (b) *init*, (c) *d-NOR*, and (d) *store*.

discharge path of latch, inhibiting the functionality of Set/Reset and preserving the original Q values (case 1.2 and 2.2 in the figure).

As we only use the existing circuitry of the bitline buffers (Figure 5(a)), this adds no hardware overhead. We name this as *computing latch* scheme. This approach is conceptually similar to Parabit [24], but it differs in that the proposed computing latch scheme leverages bidirectional data communication between the L1 and L2 latches in a modern bitline buffer [12, 14]. This enables more efficient Boolean-complete logic operations as detailed in Section 5.

4.2.2 Local Inter-Bitline Computing Module. One feasible way to implement inter-bitline computing operations is to add necessary logic gates across bitlines, as frequently performed with in-memory computations [5, 35, 37, 42, 61]. However, such an approach would yield a large area and energy overhead as demonstrated in commercial implementations [17, 41, 43].

In such regard, our strategy is to bundle a few bitlines (32 in the example) as a *local group*, and use the computing latch scheme. As shown in Figure 5(b), we connect L2 latches from each bitline buffer in a local group with a DC-interconnect which we refer to as *local module*. This will provide the same functionality from intra-bitline operations. When inter-bitline computing is not utilized, the path between L2 latch and DC-interconnect for local inter-bitline computing is disconnected by setting WEN=0.

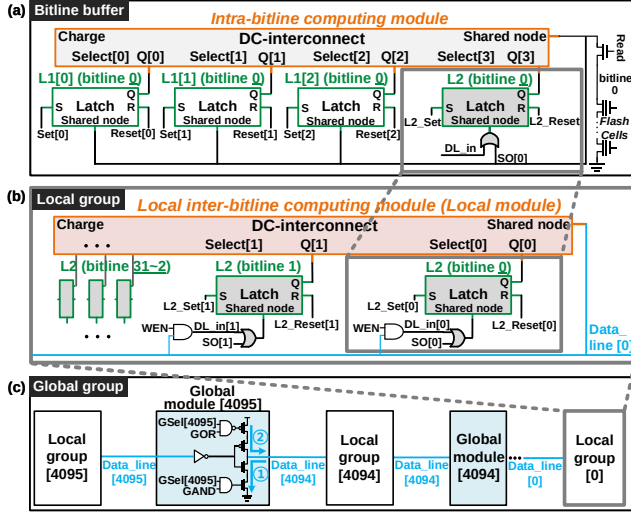


Figure 5: Hierarchical structure of CrossBit: (a) *intra-bitline*, (b) *local inter-bitline*, and (c) *global inter-bitline* operations.

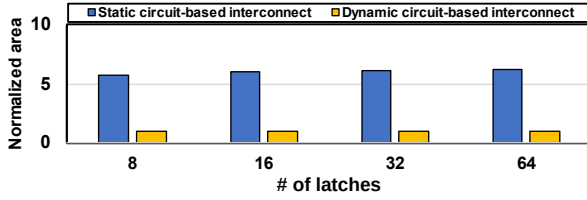


Figure 6: Area of interconnect using static circuit and dynamic circuit for local inter-bitline computing module.

The overhead is two transistors per bitline with one extra transistor for the DC-interconnect, and two logic gates per bitline for the WEN signal and SO signal from the intra-bitline computation. This is significantly cheaper than its static logic counterpart by reusing the dynamic circuit-based computing latch implementation. Figure 6 shows the high area efficiency of a local module implemented with dynamic circuits compared to one using typical static circuits, both designed with the minimal size.

4.2.3 Global Inter-Bitline Computing Module. A plane comprises a few hundreds of thousands of bitlines. Therefore, relying on the single *local module* for all bitlines is infeasible due to 1) the large capacitance of Shared_node caused by the long wire and 2) loss of parallelism as it allows only one operation per *local group*.

Thus, we devise *global modules* that integrate the results of *local groups* (*Local_group*[0], *Local_group*[1], ...) reflected on *Data_line*, the wire name corresponds to the Shared_node port of *local module*. Blue boxes in Figure 5 (c) represent the global modules. Tri-state inverter style dynamic-logic (16 transistors per module) is added between two *local groups*, reducing the transistor count from the conventional static circuit by 1.4×. It supports two operations:

- **Group AND (Figure 5 (c), ①):** When GAND=1, GOR=0, and GSEL=1, *global module* functions as an AND gate. In this mode, if *Data_line*[4095]=1, both the pull-down path(①) and pull-up

path(②) are deactivated, preserving the next data *Data_line*[4094]. On the other hand, if *Data_line*[4095]=0, ① is activated and it resets *Data_line*[4094] to 0. It sequentially resets the *Data_lines* of the following *local groups* (*Data_line*[4093], *Data_line*[4092], ...) to 0. This allows the last *local group*'s *Data_line* to store the group AND result of all *local groups*.

- **Group OR (Figure 5 (c), ②):** When GAND=0, GOR=1, and GSEL=1, *global module* functions as an OR gate. In this case, if *Data_line*[4095]=1, ② is activated and charges *Data_line*[4094] to 1. Subsequently, following *Data_lines* (*Data_line*[4093], *Data_line*[4092], ...) sequentially become 1. If *Data_line*[4095]=0, it does not affect the next *local group* *Data_line* due to deactivated pull-up and pull-down. This stores the group OR result of all *local groups* in the last *local group*'s *Data_line*.

4.3 Design considerations

4.3.1 Hierarchical Architecture. To enhance parallelism, the hierarchical approach was chosen over direct connection of the whole bitlines. The direct connection restricts parallel operations, whereas dividing bitlines into *local groups* allows for increased parallelism to the number of *local groups* with minimal additional area overhead of *global inter-bitline modules* to enable communication between local modules. As CrossBit organizes 32 different bitlines as a *local group* and 1 page is 16 KiB in general, 4096 *local groups* compute in parallel in a plane, sharing the control line for bitlines with the same index in different *local group*. When there are 256 planes in one SSD, 1048576 *local groups* compute in parallel. With direct connections, achieving the same level of parallelism would incur unacceptable area overhead. If area overhead is acceptable, adopting more hierarchies (e.g., a tree-style) can improve merging across local groups. However, we focused on an area-efficient design where a small global module sequentially connects adjacent local groups within a plane.

4.3.2 Bitline-Level IFP Unit. CrossBit has computing units at the bitline buffer (i.e., bitline-level IFP). It exploits high bitline-level parallelism as a page of data (typically 16KiB) is read from the cell array simultaneously. Alternatively, a computing unit can be placed after column multiplexer attached to bitline buffer (i.e., plane-level IFP). This location relaxes physical design constraints. However, column multiplexer fetches only 16B per cycle from bitline buffers, resulting in an additional 8us latency to transfer a page to IFP unit. CrossBit achieves 1.3×/1.2× speedup over plane-level IFP for pattern match/range query, respectively, which are fundamental database queries. For plane-level IFP, we assumed a highly optimized accelerator for the target application that computes 16B fetched data in a cycle.

4.3.3 Physical Layout. We designed CrossBit considering compatibility with industrial NAND flash and efficiency of computation. There are multiple L1 latches (i.e., L1[0], L1[1], L1[2]) and a L2 latch in each bitline buffer. L1 latches with the same index (e.g., L1[0]s) from different bitline buffers or L2 latches from different bitline buffers are placed together in conventional layout. We use L2 latch which is located at the outermost edge of conventional bitline buffer for inter-bitline data communication to minimize the interconnect length and also simplify the integration of additional

modules. For this reason, we only allow data movement between L2 latches in different bitline buffers for inter-bitline communication.

5 CrossBit Programming Method

5.1 CrossBit Primitive Functions

To efficiently leverage CrossBit through programming, we introduce two circuitry-aware primitive functions (*prims*), *prim_OR* and *prim_AND*, serving as building blocks for both intra- and local inter-bitline operations. It replaces conventional AND/OR/NOT to reduce computing steps. *prim_OR* sequentially executes *init*, *d-NOR*, and *store.case1* (Figure 4), expressed by the logic equation $dst = dst + (src1 + \dots + srcN)'$, where *dst*, *srcX*, and *N* represent the destination value, the source value stored in page buffer latch *X*, and the number of selected latches in *d-NOR*, respectively. *prim_AND*, on the other hand, sequentially executes the basic operations *init*, *d-NOR*, and *store.case2* (Figure 4), leading to the logic equation $dst = dst \cdot (src1 + \dots + srcN)$. If *N* is 0, $(src1 + \dots + srcN) = 0$. As a simple example, we can represent data transfer between latches, where $dst = src$, using *prims*. First, *prim_OR* (*N*=0) computes $dst = dst + (0)'$, making $dst = 1$. Next, *prim_AND* computes $dst = dst \cdot (src)$ (i.e., $dst = 1 \cdot (src)$), which results in $dst = src$.

The set of the two *prims* is Boolean-complete: For *N*=0, *prim_AND* performs $dst = dst \cdot (0)$, resulting in $dst = 0$. Then *prim_OR* performs $dst = 0 + (src1 + \dots + srcN)'$. It results in $dst = (src1 + \dots + srcN)'$, which is NOR between the source values, *srcX*. As NOR is Boolean-complete in itself, CrossBit programmed with the two *prims* is also Boolean-complete. Given the widespread use of logic optimizers, which convert and optimize logic with given primitive functions, designing one that converts AND/OR/NOT-based logic into *prims*-based logic appears highly feasible [3, 4, 69, 75]. However, in this study, we conducted manual conversion, deferring development of the logic optimizer to future work.

5.2 Optimizations

In compiling programs with *prims* into CrossBit control signals, further optimization is achievable by eliminating redundant signals and parallelizing independent signals. Three optimization techniques: *d-NOR-skip*, *subset-skip*, and *parallel-prim* effectively reduce computation steps, as they frequently appear in practical scenarios.

d-NOR-skip is applicable when *d-NOR* does not select any latch, allowing *prims* to bypass *d-NOR*. Furthermore, *d-NOR-skip* enables simultaneous signaling of *init* and *store*. It produces the same result as sequential *init* and *store* because an incomplete *init* naturally prevents *store* due to *Shared_node* being 0, disconnecting the pull-down path of the destination page buffer latch similar to Figure 4 (d) case 1.2/2.2. Consequently, *d-NOR-skip* reduces computing steps of a *prim* from 3 to 1.

subset-skip bypasses *init* in a *prim* when the selection of page buffer latches in *d-NOR* of the current *prim* encompasses all selections in *d-NOR* of the preceding *prim*. This is feasible because *store* from the preceding *prim*, which is performed after *d-NOR*, does not affect the result of *d-NOR* developed in *Shared_node*. It allows the next *prim* to fully utilize the *Shared_node* state, thus reusing *init* and *d-NOR* of the preceding *prim*. This reduces the computing steps of a *prim* from 3 to 2.

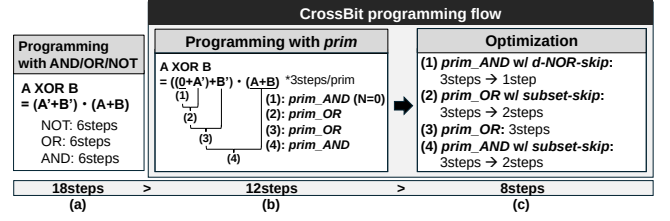


Figure 7: An example of CrossBit programming method.

parallel-prim executes current and preceding *prim* in parallel if they share identical latch selection in *d-NOR*, but different destination latch in *store*. *store* with different latch selection can run concurrently since the execution of one *store* does not corrupt the result of *d-NOR* developed in *Shared_node*.

Figure 7 introduces the programming and optimization of inter-bitline XOR as an example, used in phase 1 of in-flash error correction code (Section 6.3). While the conventional AND/OR/NOT-based programming requires 18 computing steps, programming with *prims* reduces this to 12 steps. *d-NOR-skip* and *subset-skip* further reduce the steps to 8, achieving a 2.2× overall reduction. *parallel-prim*, not employed for this example, is used in phase 2 of in-flash error correction code (Section 6.3), performing identical *prims* to different *dsts* in parallel (e.g. $dst1 = dst1 + A'$, $dst2 = dst2 + A'$).

6 CrossBit System with IF-ECC

6.1 CrossBit Workflow

This subsection outlines CrossBit's workflow after generating control signals using the programming method of Section 5. CrossBit remains compatible with ONFI, a protocol linking SSD controller and NAND flash [58]. First, SSD controller transfers the control signal through channels and store them in additional 40KiB SRAM within control logic of NAND flash die. The 40KiB SRAM can store all control signals for operations covered in this paper. IF-ECC, pattern match query, range query, multiplication, and addition require 8.5 KiB, 0.2 KiB, 2.7 KiB, 16.8 KiB, and 5.0 KiB, respectively, consuming 33.2 KiB in total. This SRAM occupies only 0.3% of die area as demonstrated in Section 8.5.

CrossBit can support complex applications with a fixed-size SRAM in two ways. First, for larger applications whose control signals exceed the SRAM capacity, the control signals can be streamed from the controller DRAM. Doing so does not affect performance because for each CrossBit basic operation (i.e., *init*, *dnor*, *store*), control signal transfer (latency=5 ns) can be completely hidden by the computation (7 ns). Compared to reading control signals from SRAM, transferring the entire signals slightly increases the energy consumption by 12% on average across all five applications we evaluate, which would not significantly affect CrossBit's energy benefits over existing techniques (2.5× on average) as shown in Section 8.3. Second, CrossBit can increase the number of *effective* control signals stored in SRAM via compression. For example, when fewer than half of the bitline buffers (i.e., < 16) in a local group are active during a computing cycle, we can compress the control signals by omitting ones for inactive bitline buffers (reducing at least 2×17 bits) and sending extra 33 bits: 1 bit to indicate this mode,

32 bits to specify active bitline buffers. Such compression reduces the amount of control signals of IF-ECC by 1.46×. We leave the development of more efficient compression methods to future work.

To indicate the start and end of the control signal, two opcodes 33h and 34h, are set among ONFI vendor-specific opcodes. Then, external input data is transferred from SSD controller to NAND flash die and stored in page buffer by toggling ONFI data transfer signal WE#, resembling conventional write data streaming. To broadcast control signal and external input data through channels, ONFI chip-enable signal CE# is set high for all dies. Subsequently, SSD controller issues a modified cache read command with vendor specific ONFI opcode 36h (Section 6.2). This command sequentially performs cell read, in-flash error correction for MLC (Section 6.3), and target application. Then, it transfers computation results in page buffer to SSD controller by toggling ONFI read out signal RE#.

6.2 Consistency with Memory Operation

We modified ONFI command, cache read, to implement *intra-bitline* and *inter-bitline* operations. It allows to utilize existing FSM-based scheduler module in NAND flash by introducing new states. Cache read consists of two steps. First, it moves the data stored in L1[0] latch to L2 latch, consuming tRCBSY latency. Second, it simultaneously transmits data to SSD controller and reads cell data. CrossBit replaces the latch operations in the first step of cache read with computations, consuming latency of tRCBSY_CrossBit. While this increases the page buffer operation time from tRCBSY to tRCBSY_CrossBit, it enables arbitrary logic computations such as IF-ECC (Section 6.3), pattern match queries (Section 7.1), and range queries (Section 7.2). The tRCBSY_CrossBit for these operations are 5.714μs, 299.7ns, and 3.094μs, respectively. However, CrossBit takes advantage in the overall execution time by computing inside NAND flash, dramatically reducing the amount of data transfer.

6.3 In-Flash Error Correction (IF-ECC)

Enhanced MLC Programming (EMP): EMP, derived from Enhanced SLC Mode Programming (ESP) of Flash-Cosmos, improves the reliability of write on MLC by tightening the distribution of each data level with reduced ISPP voltage step to enhance the read margin between data levels as shown in Figure 8 (a). While ESP enables the use of SLC for IFP, EMP alone cannot achieve sufficient BER due to the reduced read margin of MLC [60]. Therefore, we combine EMP with IF-ECC to improve BER to use MLC in IFP.

IF-ECC: As ECC mostly relies on XOR operations, CrossBit has a unique opportunity to perform in-flash ECC (IF-ECC) using its inter-bitline functionality. Each *local group* adds six bitlines to store parity bits for it. CrossBit applies Hamming code [40, 63] to detect and correct errors within each *local group*. Stronger ECC algorithms used in modern SSDs like BCH or LDPC can also be processed due to Boolean-completeness of CrossBit. However, they are not as cost-efficient as Hamming code due to the high computational complexity to implement across all NAND flash planes in parallel [15], making it not feasible for IFP. Utilizing the stronger ECC module in SSD controller is inefficient as well. It requires transferring cell data to the controller and sending it back to flash plane for processing, which consumes substantial energy and latency. Therefore, we opted for the more lightweight ECC algorithm with more parity

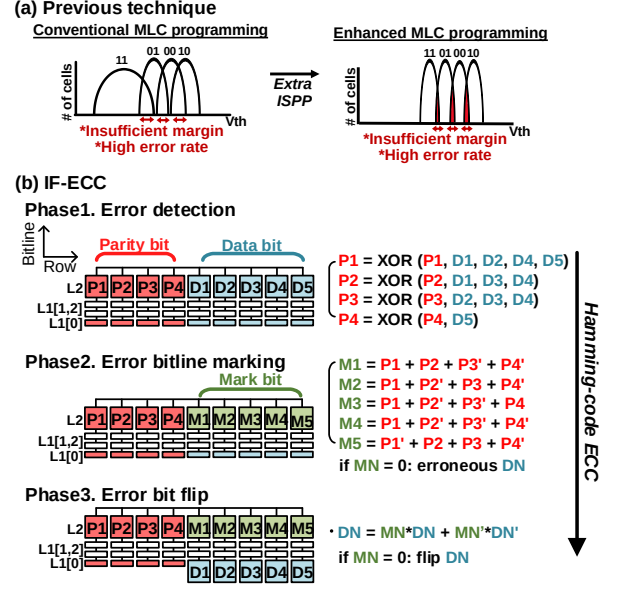


Figure 8: (a) Previous error reduction technique and (b) In-flash error correction (IF-ECC) process using CrossBit.

bits, offering a significant capacity gain over SLC-based IFP (Section 8.2). As parity bits of conventional ECC and IF-ECC are different, we separate physical address space for IFP data and place additional ECC encoding module at the controller.

CrossBit executes IF-ECC with 3 phases, performing the logic of Figure 8 (b) in parallel within each *local group*. We reduced the number of bitlines for clarity in explanation while preserving the underlying principle. Phase 1 involves inter-bitline XOR on the predefined sets of parity bit (PN) and data bit (DN), each stored in separate bitlines, as defined by Hamming code algorithm [40, 63]. The XOR results are overwritten to PN. For example, $P1 = \text{XOR}(P1, D1, D2, D4, D5)$ and $P2 = \text{XOR}(P2, D1, D3, D4)$. In phase 2, CrossBit identifies the error bitline with the *mark bit* of DN (MN)=0. MN is uniquely indicated by the results of phase 1, using inter-bitline operations based on hamming code algorithm. For example, $M1 = P1 + P2 + P3' + P4'$ and $M2 = P1 + P2' + P3 + P4'$. Phase 3 flips the cell data with intra-bitline operations if the bitline is marked as MN=0, indicating erroneous DN, by performing $DN = MN \cdot DN + MN' \cdot DN'$. After correcting errors, CrossBit seamlessly proceeds to other applications without additional operations, as IF-ECC replaces erroneous data with error-corrected data in the same location (L1[0]) where it was originally stored after cell read. IF-ECC can be efficiently implemented using the proposed programming method (Section 5). CrossBit consumes 2217, 1485 and 783 computing steps for programming with AND/OR/NOT, *prims* and *prim* with optimization, respectively.

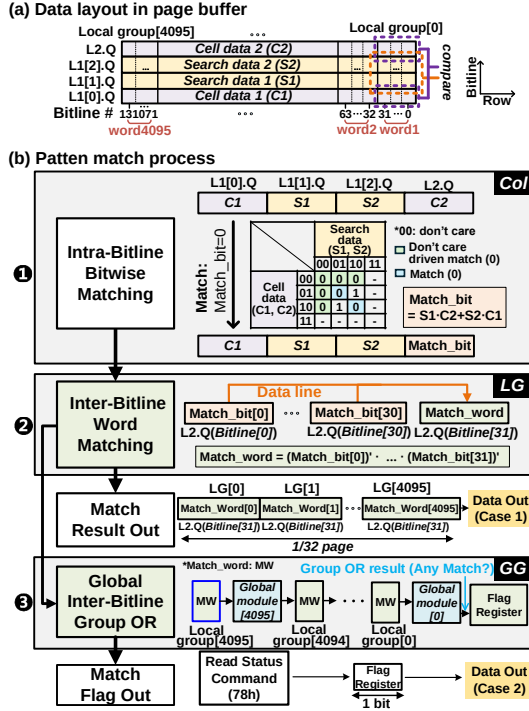


Figure 9: Pattern match query using CrossBit.

7 Database Applications with CrossBit

7.1 Pattern Match Query

Pattern match queries involve searching for words transferred from off-chip source, among cell data stored in NAND flash. Intra-bitline operation performs bitwise matching for each bit position of words. Then, inter-bitline operations merge bitwise matching results for word-level matching. Figure 9 (a) illustrates the data layout in page buffer for cell data read out from cell array, and the search data from the external source. We represent data of each bit position with two bits, S1/S2 for search bit and C1/C2 for cell data, to represent logical 1, 0, and don't care values as 10, 01, and 00, respectively.

CrossBit executes pattern match query as shown in Figure 9 (b). Initially, intra-bitline bitwise matching (①) performs logic operation representing the bitwise matching truth table, expressed as $Match_bit = S1 \cdot C2 + S2 \cdot C1$, while $Match_bit = 0$ indicates match in a bit position. Subsequently, inter-bitline word matching (②), denoted as $Match_word = (Match_bit[0])' \cdot (Match_bit[1])' \cdot \dots \cdot (Match_bit[31])'$, verifies if $Match_bit = 0$ across all bitlines in a local group, signifying a match in word. Lastly, CrossBit identifies word match cases by extracting $Match_words$ of each local group externally (Case 1). Compared to non-IFP systems this reduces the data transfer from 32 bits to a single bit per word.

Furthermore, we can determine if a match exists in any local group inside a chip using group OR (③). Flag register of conventional NAND flash, optimized for small-scale data transfer, stores the 1-bit group OR result. ONFI Read status command (78h) transfers this data to off-chip controller (Case 2). For efficient search, the two cases can be combined: Case 2 checks for a match, and

Case 1 is executed only if a match is found. CrossBit consumes computing steps of 22, 13, and 12, respectively for programming with AND/OR/NOT, *prims*, and *prims* with optimization.

7.2 Range Query

Range queries determine whether data lies within the specified range. CrossBit processes range queries with numerical comparisons between two unsigned integers. When A is the stored data in memory and B is the range data from host, there can be two types of query: $A > B?$ and $A < B?$. By inverting the result of the two queries, $A \leq B?$ and $A \geq B?$ can be answered. Various queries can be answered with their combinations (e.g. $b_1 < A \leq b_2?$, $b_1 \leq A \leq b_2?$). 2's complement data can also be processed by combining them.

The comparison process of two unsigned integer numbers are depicted in Figure 10. A_0 is the data from memory and B_0 is the input range from host. In the first phase, CrossBit *eliminates the common 1* of A_0, B_0 in the same bit position to eliminate the unneeded data for the comparison. CrossBit conducts it by performing $A1 = A0 \cdot B0'$ and $B1 = A0' \cdot B0$ with *intra-bitline operations*. In phase 2, CrossBit performs *unidirectional OR* on $A1$ s from MSB to LSB with *inter-bitline operations*, resulting in $A2$. This process leaves only the information of the first 1 coming out from MSB, eliminating the information of the 1's coming out after the first 1. In phase 3, CrossBit performs *bitwise $A > B?$ (or $A < B?$)*. For this, CrossBit conducts $C = A2 \cdot B2'$ (or $C = A2' \cdot B2$) with *intra-bitline operations*. In phase 4, CrossBit performs *OR* between C s with *inter-bitline operations* to get the final result. It returns the result 1 when A_0 falls within the range B_0 , and 0 otherwise. By programming with AND/OR/NOT, *prims*, and *prims* with optimization, CrossBit performs range query in 591, 270, and 245 computing steps, respectively.

7.3 Join Query

Join query is one of the major bottlenecks of database management [7, 8, 66]. It uniquely matches each record in a fact table with one of dimension tables. We adopt a widely-used partitioned join algorithm [8] for CrossBit, by partitioning in SSD controller and storing the partitions in NAND flash. As dimension table is much smaller than fact table [9, 59], CrossBit broadcasts the partitioned dimension table to NAND flash and searches for match with pattern match query. To reduce the search data transfer overhead, CrossBit efficiently copies a single 32-bit search data from SSD controller to all local groups. To store '1' in a specific bit position across all local groups, CrossBit performs *prim_OR* to the target bit position with $N=0$, resulting in $dst=1$, as they share the same control line. Conversely, to store '0', CrossBit executes *prim_AND* with $N=0$.

8 Evaluation

8.1 Experimental Methods

BER Calculation: To evaluate the error reduction capability of IF-ECC, we compared the MLC BERs after IF-ECC (CrossBit) with those obtained using EMP only. We used the Gaussian distribution parameters of flash cell data from [11] to determine the probability of data corruption. For BER after IF-ECC, we computed the probability of having more than one error in a local group because Hamming-code algorithm that IF-ECC utilizes can correct at most 1 error [40, 57, 63]. The calculation method is detailed in [11, 53].

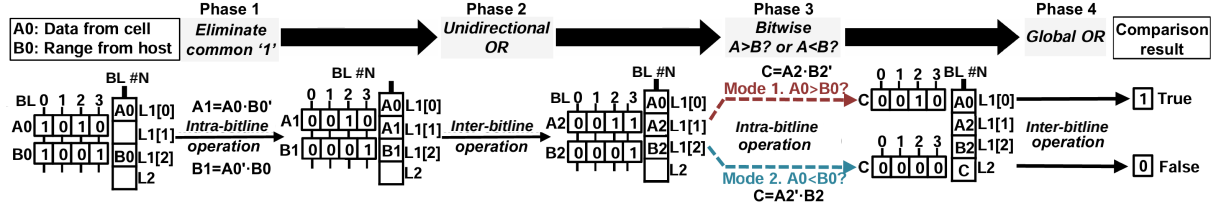


Figure 10: Unsigned integer comparison process of CrossBit for range query.

Table 1: Computing system configuration

SSD	NAND flash	8 channels, 8 dies/channel, 4 planes/die, 1024 blocks/plane, 1024 pages/block, 16KiB/page
	DRAM	DDR4-1800, 1GB
	Latency	SLC: $t_R = 20\mu s$, $t_{PROG} = 200\mu s$ MLC: $t_R = 25\mu s$, $t_{PROG} = 500\mu s$
	Bandwidth	Host interface: 8GB/s (PCIe Gen4 $\times 4$ lanes), Channel: 16GB/s (NV-DDR2 $\times 8$ channels)
	technology	150nm CMOS
	Supply	$V_{cc} = 2.35V$, $V_{ccq} = 1.2V$
FPGA	274,080 LUTs, 548,160 Flip-Flops, 200MHz	
DRAM	Samsung DDR4 PC4-25600 32GB [22]	
CPU	Intel i7-12700K 3.60GHz [32]	

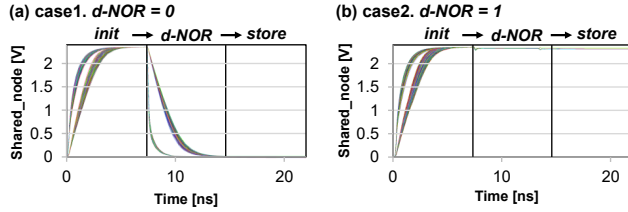


Figure 11: CrossBit operations under variation.

Latency and Energy Simulation: We compare CrossBit with previous IFPs (ParaBit, ParaBit-LocFree, Flash-Cosmos, and Ares-Flash), ISP, and host processing (HP). HP performs computation within the CPU. We include IF-ECC overhead for CrossBit and Ares-Flash, which supports inter-bitline computations. For Flash-Cosmos, which computes operands stored in flash cells, we allocate four layers of cell array to store external input.

Table 1 outlines the hardware configurations for evaluation. The SSD configuration is based on [14] and [21]. Parameters of [58] and [51] are utilized to estimate energy consumption of NAND flash. For SSD latency, we utilize a SOTA SSD simulator, MQSim [68]. The energy consumption of NAND flash is measured using Micron Parallel NAND System Power Calculator [52]. Transistor-level latency and energy consumption for CrossBit operations are simulated with HSPICE [67]. Parasitic resistance and capacitance of wire for post-layout SPICE simulations were extracted using xRC [65]. For robust computation and simplified control, we fix the *1 basic operation time* to 7.3ns, the maximum among all basic operations. This indicates that control signals such as SET/RESET are active for 7.3ns with $V_{cc}=2.35V$. We validated the *1 basic operation time* with 1,000-point Monte-Carlo simulations using HSPICE (Figure 11). ISP accelerator latency is measured

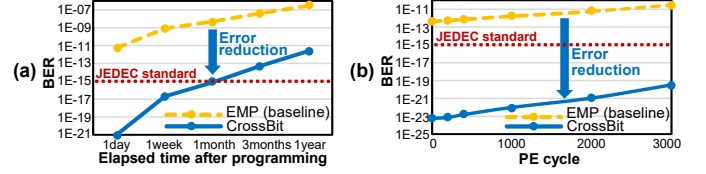


Figure 12: BER with (a) different elapsed times (P/E cycle=2K) and (b) different P/E cycles (elapsed time=1day).

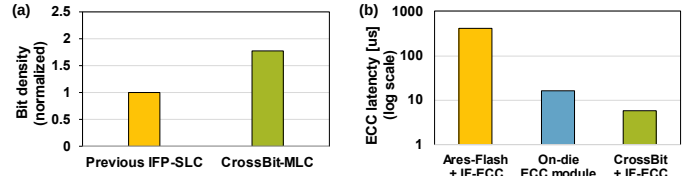


Figure 13: (a) Bit density of IFP with SLC and CrossBit with MLC. (b) ECC latency comparison.

with FPGA [74]. DRAM latency is measured using DRAMsim3 [46]. 3.60GHz Intel i7 CPU is used for performance of HP.

8.2 IF-ECC Evaluation Results

BER of MLC for IFP: Figure 12 shows the BER of MLC with IF-ECC, labeled as CrossBit, compared to the baseline, EMP. In Figure 12 (a), with program-and-erase (P/E) cycle fixed at 2K and varying elapsed time after programming, the averaged BER reduction is $1.1E+9\times$. In Figure 12 (b), with varying P/E cycles and elapsed time fixed at 1 day, the averaged BER reduction is $3.7E+10\times$.

JEDEC standard of uncorrectable BER (UBER), the BER after ECC, is 10^{-15} for client-class SSDs and 10^{-16} for enterprise-class SSDs [1]. However, real-world SSDs have shown UBERs between 10^{-9} to 10^{-14} , not meeting the JEDEC standard [23, 49, 56]. Given this range, MLC corrected by IF-ECC can be used in the real-world for the time and P/E cycle of Figure 12, while one week is the maximum for EMP. Within the JEDEC standard, IF-ECC provides data retention up to one month at 2K P/E cycle. In contrast, EMP does not meet the standard even immediately after programming.²

Improvement in Bit Density: Figure 13(a) shows that CrossBit achieves a $1.8\times$ higher bit density with MLC than the previous

²The JEDEC BER standard evaluates the average UBER of SSD lifetime, while our experimental result represents the instantaneous UBER. Since BER increases with time and P/E cycles, our instantaneous BER result is always greater than the average BER measurement condition of the JEDEC method.

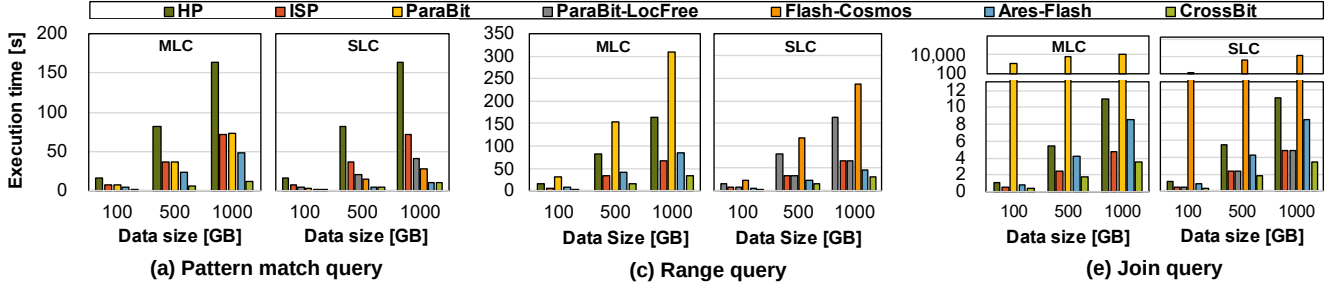


Figure 14: Execution time of various architectures for (a) pattern match query, (b) range query, and (c) join query.

IFPs using SLC. This benefits monetary cost, energy efficiency, and performance for data-intensive workloads. The reason for the slightly lower bit density gain compared to the doubled cell capacity (SLC to MLC) is the addition of parity bits for IF-ECC.

Performance of IF-ECC: Figure 13(b) compares the latency of IF-ECC processed with CrossBit, Ares-Flash, and on-die ECC module in a plane. The on-die ECC module can compute ECC in 1 cycle with dedicated logic, but it needs to fetch 16B per cycle via column multiplexer attached to page buffer consuming additional latency. IF-ECC with CrossBit achieves 71× speedup over IF-ECC with Ares-Flash, as Ares-Flash provides restricted inter-BL computation. IF-ECC processed with CrossBit achieves 2.8× speedup over on-die ECC module which requires additional data transfer latency.

8.3 Database Query Acceleration

This subsection presents evaluation for fundamental database operations along with the end-to-end SSB [59] queries.

Evaluation Scenario: To evaluate pattern match query, we assume the following. Each 32-bit data is stored in a column-store manner, known for its efficiency in database [16]. For ParaBit and Flash-Cosmos, 32 bit data are stored in a transposed manner in a bitline to utilize their computing scheme, while others adhere to the convention of storing data in the same row. Each system scans database until it finds a match with input data [29]. To evaluate range query, we simulated a color recognition scenario [10], using image with four 8-bit channel data per pixel [62, 71]. Data layout is consistent with pattern match except for the data length. Each color has its specific value range for each channel, and the system recognizes color when all channel values of a pixel fall within this range. For join query, we applied the process of CrossBit to other IFPs, conventional hash table-based search method which is faster in general [7, 8, 66] for others. For the table size, we used join cases from Star-Schema Benchmark (SSB) with scale factor of 1000 [59] and averaged them. The queries of SSB are the combinations of pattern match query, range query, join, and arithmetic operations.

Execution Time: For pattern match query, with MLC, CrossBit delivers a geomean speedup of 5.8×/3.9× over ParaBit/Ares-Flash. Under SLC, CrossBit achieves a geomean speedup of 4.2×/2.9×/1.0× over ParaBit-LocFree/Flash-Cosmos/Ares-Flash, respectively. For range query, CrossBit achieves a geomean speedup of 9.3×/2.4× over ParaBit/Ares-Flash with MLC. For SLC, CrossBit achieves a geomean speedup of 2.2×/7.8×/1.5× over ParaBit-LocFree/Flash-Cosmos/Ares-Flash, respectively. For join, with MLC, CrossBit

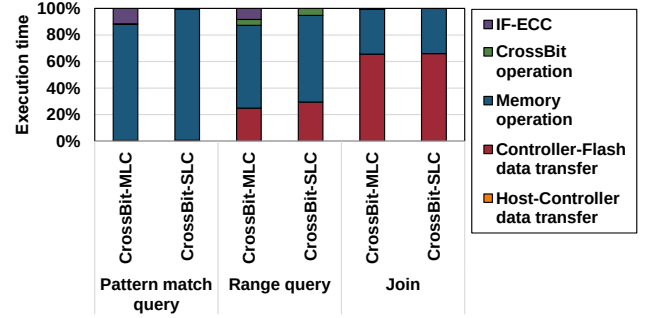


Figure 15: Execution time breakdown of CrossBit.

shows a geomean speedup of 3.7K×/2.4× over ParaBit/Ares-Flash and under SLC, CrossBit achieves a geomean speedup of 1.4×/350×/2.4× over ParaBit-LocFree/Flash-Cosmos/Ares-Flash, respectively. Inter-bitline operations takes up 93.1%/78.4%/94.8% of CrossBit operations (i.e., intra- and inter-bitline computation) for IF-ECC/pattern match query/range query, respectively. This underscores the critical role of inter-bitline operations in complex computations.

CrossBit excels over Ares-Flash by supporting more flexible inter-bitline computations. CrossBit outperforms Flash-Cosmos by avoiding write overhead, which becomes severe in range and join queries due to larger inputs. In particular, the performance of Flash-Cosmos for join is significantly degraded as each fact table entry in a partition has to be compared with the entire dimension table in the same partition after the cell write. CrossBit outperforms ParaBit-LocFree because ParaBit-LocFree supports limited intra-bitline operation, offloading the remaining computation to the SSD controller. The performances of CrossBit-MLC and CrossBit-SLC are comparable, as IF-ECC only accounts for a short execution time as shown in Figure 15. This is because reading data from flash cells to page buffer takes significantly longer than IF-ECC.

Figure 16 shows the execution time for processing queries of SSB. The geometric mean of speedup among different queries is 1.7. ParaBit and Flash-Cosmos experience performance degradation due to the significant cell programming overhead. Additionally, CrossBit, as a Boolean-complete logic accelerator, supports addition, subtraction, and multiplication required in SSB queries. Inter-bitline operations occupy an average portion of 64% of CrossBit operations across the entire set of SSB queries.

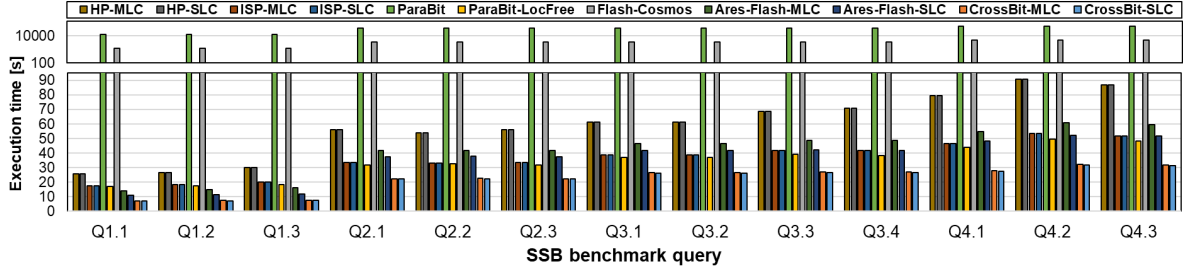


Figure 16: Execution time of various computing architectures for processing SSB (scale factor: 1000) benchmark queries.

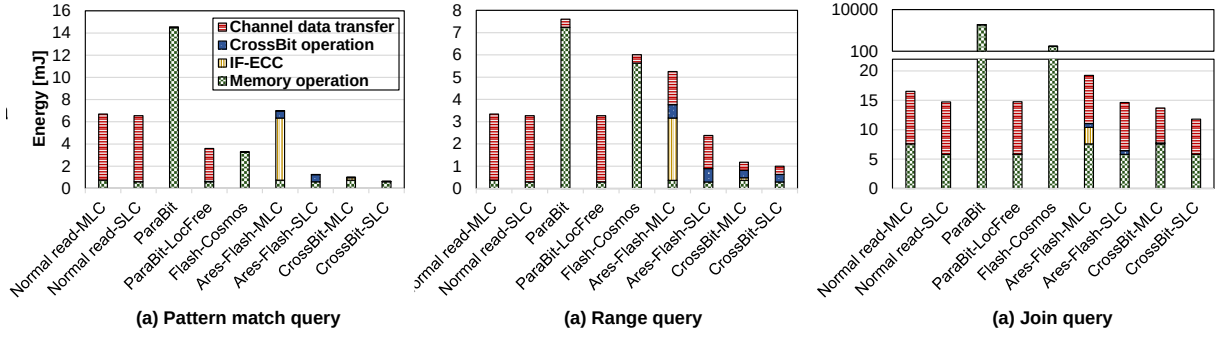


Figure 17: Comparison of energy consumption for processing (a) pattern match query, (b) range query, and (c) join.

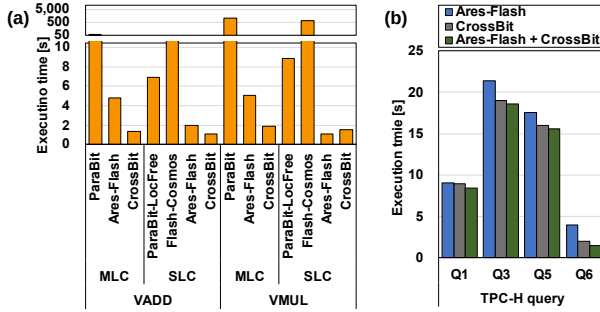


Figure 18: Execution time of (a) VADD and VMUL, (b) TPC-H.

Energy Consumption: As shown in Figure 17, CrossBit exhibits the lowest energy consumption in NAND flash, due to reduced data transfer, cell write overhead, and efficient inter-bitline computation. For pattern match queries, with MLC, CrossBit consumes $15\times/7.0\times$ less energy than ParaBit/Ares-Flash, and under SLC, CrossBit consumes $5.8\times/5.3\times/2.0\times$ less energy than ParaBit-LocFree/Flash-Cosmos/Ares-Flash, respectively. Similar trends are found from range queries and join queries with both MLC and SLC, demonstrating the significant energy savings from CrossBit.

8.4 Acceleration of Arithmetic Operations

Figure 18(a) shows the evaluation results for vector addition (VADD) and vector multiplication (VMUL) using a 100GB 8-bit vector dataset. For VADD, CrossBit and Ares-Flash perform similar ripple-carry addition by transferring one carry bit to the adjacent bitline at a time.

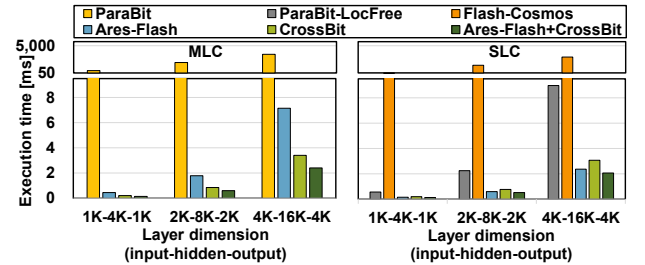


Figure 19: Execution time for MLP.

However, CrossBit reduces computing cycles using its programming method, achieving $3.6\times$ and $1.9\times$ speedup over Ares-Flash with MLC and SLC, respectively. The speedup is greater with MLC due to IF-ECC processed with limited inter-bitline computation of Ares-Flash. Without programming method, CrossBit gains only $1.1\times$ speedup over Ares-Flash for SLC. For VMUL, CrossBit sequentially shifts each bit data stored as bitline buffers using a shared transfer node (Section 4.2). In contrast, Ares-Flash shifts all bit data simultaneously. As a result, with SLC, CrossBit is $1.4\times$ slower than Ares-Flash. However, under MLC usage, due to the ECC overhead in Ares-Flash, CrossBit achieves $2.7\times$ speedup over Ares-Flash.

Meanwhile, implementing both CrossBit and Ares-Flash circuits in a flash array is straightforward as the inter-bitline connections of each architecture do not interfere with each other and consume minimal area. CrossBit connects L2 latches, while Ares-Flash designs data transfer path from L2 latch to L1 latch. This integration of the two architectures can offer better performance. Figure 18(b) shows

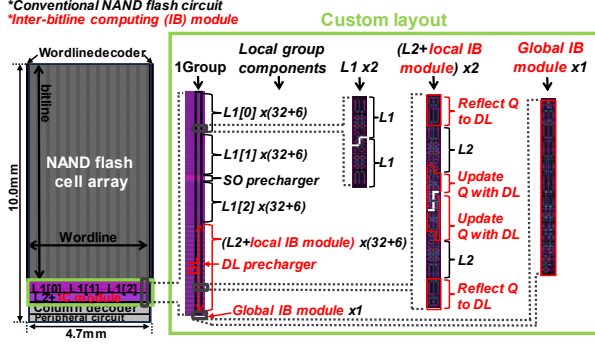


Figure 20: Custom layout of CrossBit (green box) with proportionally sized virtual boxes illustrating other flash circuits.

that for TPC-H [9] queries involving frequent arithmetic operations, the integration of CrossBit and Ares-Flash achieves $1.1 \times 1.4 \times$ speedup over standalone CrossBit/Ares-Flash, respectively. Similar results can be found from Figure 19, where we evaluated multi-layer perceptron (MLP) with varying dimensions by processing its MAC operations. We used 8-bit integer for input and weight, and 32-bit integer for accumulation. Again, the integration of Ares-Flash (i.e., specialized in arithmetic operations) and CrossBit (i.e., designed for general purpose) can enhance the performance, achieving $1.4 \times 3.0 \times$ speedup in MLC and $1.5 \times 1.1 \times$ speedup in SLC, over standalone CrossBit/Ares-Flash, respectively. As MLP is a crucial component in machine learning such as language [72] or vision [28, 70] models, it implies the high potential for broader workloads.

8.5 Area Overhead

To estimate area overhead, we manually drew a custom layout (the green box in Figure 20). The same figure includes boxes indicating the location and physical dimensions of the other components in a commercial 3D NAND flash [44]. To reflect the characteristics of industrial products, we used transistors with 150nm gate length similar to those used in modern 3D NAND peripheral circuits. Note that such a relatively long gate length is commonly used in modern 3D NAND products due to technological and economic reasons, including the need to satisfy thermal constraints from fabricating flash cell array on the same wafer [31, 39, 64]. Meanwhile, the industry recently has been actively pursuing 3D NAND structures that use separate wafers for stacking the cell array and periphery [30, 45, 64]. In these structures, more advanced logic technology is expected to be used. In Section 9, we investigate the effect of more advanced technology nodes. Figure 21 shows that the additional area for CrossBit is 2.2% of the total die area.

9 Discussion

Technology Scaling. As described in Section 8.1, scaled logic technologies are expected to be used for peripheral circuits in the advanced NAND array. To evaluate the effect on CrossBit, we also conducted HSPICE simulations in a 28nm CMOS technology. The results show that the technology scaling improves latency and energy consumption of computing modules by $2.7 \times$ and $12.0 \times$, respectively, in geomean across the applications evaluated throughout the paper.

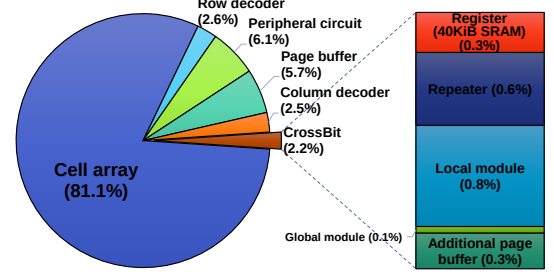


Figure 21: Area overhead of CrossBit

Relationship with Existing ECC. IF-ECC has potential to improve performance of conventional NAND flash. However, we dedicate a larger number of parity bits than conventional off-chip ECCs, such as LDPC or BCH to meet the BER target with IF-ECC. As a result, CrossBit achieves $1.8 \times$ higher bit density with MLC than SLC. While this improvement is significant for IFP, where off-chip ECC cannot be applied, using stronger off-chip ECC with less bit density loss is more desirable for conventional memory operations. It necessitates separating the physical address space between data for IFP and data for conventional memory access.

Data Layout. While CrossBit supports computation of arbitrary data layouts in a plane with intra- and inter-bitline operations, it is true that the performance is dependent on the data layout. Similar to other IFP techniques, the performance is maximized when the data are organized in a dense array, such as columnar databases as opposed to row databases as demonstrated in this work. Because columnar formats are getting popular especially for OLAP workloads, we believe that CrossBit fits the recent trend. Even when row-major format must be favored, the conversion is one-time overhead which requires a single access.

10 Conclusion

CrossBit introduces a new IFP architecture focused on enabling efficient inter-bitline operations with minimal hardware overhead. A key strength of CrossBit lies in its versatility to handle applications previously outside the scope of IFP architectures, which were limited to intra-bitline operations. Through meticulous design and optimization, we have empowered CrossBit to accelerate computing, particularly in in-flash ECC and database queries.

Acknowledgments

This work was supported in part by Samsung Electronics, Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2023-00208606, NeuroHub+: Scheduler and Simulator for General In-Memory Neural Network Accelerators, RS-2024-00395134, DPU-Centric Datacenter Architecture for Next-Generation AI Devices, No. 2021-0-01343: Artificial Intelligence Graduate School Program (Seoul National University)), and BK21 FOUR program at Seoul National University. The EDA tool was supported by the IC Design Education Center (IDEC). (Corresponding Authors: Jinho Lee and Jae-Joon Kim).

References

- [1] 2013. *JEDEC SSD Specifications Explained*. [https://www.jedec.org/sites/default/files/Alvin_Cox%20\[Compatibility%20Mode\]_0.pdf](https://www.jedec.org/sites/default/files/Alvin_Cox%20[Compatibility%20Mode]_0.pdf)
- [2] Shaizeen Aga, Supreet Jeloka, Arun Subramaniyan, Satish Narayanasamy, David Blaauw, and Reetuparna Das. 2017. Compute caches. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 481–492.
- [3] Luca Amaru, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. 2015. Majority-inverter graph: A new paradigm for logic optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 35, 5 (2015), 806–819.
- [4] Luca Amaru, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. 2015. Majority-inverter graph: A new paradigm for logic optimization. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 35, 5 (2015), 806–819.
- [5] Hadi Asghari-Moghaddam, Young Hoon Son, Jung Ho Ahn, and Nam Sung Kim. 2016. Chameleon: Versatile and practical near-DRAM acceleration architecture for large memory systems. In *MICRO*.
- [6] Ganesh Balamurugan and Naresh R Shanbhag. 2001. The twin-transistor noise-tolerant dynamic circuit technique. *IEEE Journal of Solid-State Circuits* 36, 2 (2001), 273–280.
- [7] Cagri Balkesen, Gustavo Alonso, Jens Teubner, and M Tamer Özsu. 2013. Multi-core, main-memory joins: Sort vs. hash revisited. *Proceedings of the VLDB Endowment* 7, 1 (2013), 85–96.
- [8] Çağrı Balkesen, Jens Teubner, Gustavo Alonso, and M Tamer Özsu. 2014. Main-memory hash joins on modern processor architectures. *IEEE Transactions on Knowledge and Data Engineering* 27, 7 (2014), 1754–1766.
- [9] Peter Boncz, Thomas Neumann, and Orri Erling. 2013. TPC-H analyzed: Hidden messages and lessons learned from an influential benchmark. In *Technology Conference on Performance Evaluation and Benchmarking*. Springer, 61–76.
- [10] James Bruce, Tucker Balch, and Manuela Veloso. 2000. Fast and inexpensive color image segmentation for interactive robots. In *Proceedings. 2000 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2000)(Cat. No. 00CH37113)*, Vol. 3. IEEE, 2061–2066.
- [11] Yu Cai, Saugata Ghose, Erich F Haratsch, Yixin Luo, and Onur Mutlu. 2017. Error characterization, mitigation, and recovery in flash-memory-based solid-state drives. *Proc. IEEE* 105, 9 (2017), 1666–1704.
- [12] HM Cao, F Liu, Q Wang, ZC Du, L Jin, and ZL Huo. 2022. An efficient built-in error detection methodology with fast page-oriented data comparison in 3D NAND flash memories. *Electronics Letters* 58, 12 (2022), 483–485.
- [13] Jian Chen, Congming Gao, Youyou Lu, Yuhao Zhang, and Jiwu Shu. 2024. Ares-Flash: Efficient Parallel Integer Arithmetic Operations Using NAND Flash Memory. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1489–1503.
- [14] Jiho Cho, D Chris Kang, Jongyeol Park, Sang-Wan Nam, Jung-Ho Song, Bong-Kil Jung, Jaedong Lyu, Hogil Lee, Won-Tae Kim, Hongsoo Jeon, et al. 2021. 30.3 A 512Gb 3b/Cell 7 th-Generation 3D-NAND Flash Memory with 184MB/s Write Throughput and 2.0 Gb/s Interface. In *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 64. IEEE, 426–428.
- [15] Myungjun Chun, Jaeyong Lee, Sanggu Lee, Myungsuk Kim, and Jihong Kim. 2022. PiF: In-flash acceleration for data-intensive applications. In *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*. 106–112.
- [16] Carlos Coronel and Steven A Morris. 2022. *Database systems: design, implementation and management*. Cengage learning.
- [17] Fabrice Devaux. 2019. The true Processing In Memory accelerator. In *HCS*.
- [18] Martin Dimitrov, Karthik Kumar, Patrick Lu, Vish Viswanathan, and Thomas Willhalm. 2013. Memory system characterization of big data workloads. In *2013 IEEE International Conference on Big Data*. IEEE, 15–22.
- [19] Li Ding and Pinaki Mazumder. 2004. On circuit techniques to improve noise immunity of CMOS dynamic logic. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 12, 9 (2004), 910–925.
- [20] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramaniyan, Ravi Iyer, Dennis Sylvester, David Blaauw, and Reetuparna Das. 2018. Neural cache: Bit-serial in-cache acceleration of deep neural networks. In *2018 ACM/IEEE 45th annual international symposium on computer architecture (ISCA)*. IEEE, 383–396.
- [21] Samsung Electronics. 2024. *Samsung 990 PRO*. <https://semiconductor.samsung.com/consumer-storage/internal-ssd/990-pro/>
- [22] Samsung Electronics. 2024. *Samsung DDR4 PC4-25600 32GB*. <https://semiconductor.samsung.com/dram/ddr/ddr4/>
- [23] Emna Farjallah, Jean-Marc Armani, Valentin Gherman, and Luigi Dilillo. 2019. Improvement of the tolerated raw bit error rate in NAND flash-based SSDs with selective refresh. *Microelectronics Reliability* 96 (2019), 37–45.
- [24] Congming Gao, Xin Xin, Youyou Lu, Youtao Zhang, Jun Yang, and Jiwu Shu. 2021. Parabit: processing parallel bitwise operations in nand flash memory based ssds. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 59–70.
- [25] Mingyu Gao, Grant Ayers, and Christos Kozyrakis. 2015. Practical near-data processing for in-memory analytics frameworks. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*. IEEE, 113–124.
- [26] Boncheol Gu, Andre S Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, et al. 2016. Biscuit: A framework for near-data processing of big data workloads. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 153–165.
- [27] Saransh Gupta, Justin Morris, Mohsen Imani, Ranganathan Ramkumar, Jeffrey Yu, Aniket Tiwari, Baris Aksanli, and Tajana Šimunić Rosing. 2020. Thrifty: Training with hyperdimensional computing across flash hierarchy. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.
- [28] Kai Han, Yunhe Wang, Hanting Chen, Xinghao Chen, Jianyuan Guo, Zhenhua Liu, Yehui Tang, An Xiao, Chunjing Xu, Yixing Xu, et al. 2022. A survey on vision transformer. *IEEE transactions on pattern analysis and machine intelligence* 45, 1 (2022), 87–110.
- [29] Hao Huang, Xindong Wu, and Richard Relue. 2002. Association analysis with one scan of databases. In *2002 IEEE International Conference on Data Mining, 2002. Proceedings.* IEEE, 629–632.
- [30] Zongliang Huo, Weihua Cheng, and Simon Yang. 2022. Unleash scaling potential of 3D NAND with innovative Xtacking® architecture. In *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 254–255.
- [31] Satoshi Inaba. 2018. 3D flash memory for data-intensive applications. In *2018 IEEE International Memory Workshop (IMW)*. IEEE, 1–4.
- [32] Intel. 2022. *Intel i7-12700K 3.60GHz*. <https://www.intel.com/content/www/us/en/products/sku/134594/intel-core-i712700k-processor-25m-cache-up-to-5-00-ghz/specifications.html>
- [33] Wontaek Jung, Hyunggon Kim, Do-Bin Kim, Tae-Hyun Kim, Namhee Lee, Dongjin Shin, Minyoung Kim, Youngsik Rho, Hun-Jong Lee, Yujin Hyun, et al. 2024. 13.3 A 280-Layer 1Tb 4b/cell 3D-NAND Flash Memory with a 28.5 Gb/mm² Areal Density and a 3.2 GB/s High-Speed IO Rate. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 67. IEEE, 236–237.
- [34] Dongku Kang, Minsu Kim, Su Chang Jeon, Wontaek Jung, Jooyoung Park, Gyoosoo Choo, Dong-kyo Shim, Anil Kavala, Seung-Bum Kim, Kyung-Min Kang, et al. 2019. 13.4 A 512Gb 3-bit/Cell 3D 6 th-Generation V-NAND flash memory with 82MB/s write throughput and 1.2 Gb/s interface. In *2019 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 216–218.
- [35] Byeongho Kim, Jongwook Chung, Eojin Lee, Wonkyung Jung, Sunjung Lee, Jaewan Choi, Jaehyun Park, Minbok Wi, Sukhan Lee, and Jung Ho Ahn. 2020. MViD: Sparse Matrix-Vector Multiplication in Mobile DRAM for Accelerating Recurrent Neural Networks. *IEEE TC* (2020).
- [36] Hyunsuk Kim, Su-Jin Ahn, Yu Gyun Shin, Kyupil Lee, and Eunseung Jung. 2017. Evolution of NAND flash memory: From 2D to 3D as a storage market leader. In *2017 IEEE International Memory Workshop (IMW)*. IEEE, 1–4.
- [37] Heesu Kim, Hanmin Park, Taehyun Kim, Kwanheum Cho, Eojin Lee, Soojung Ryu, Hyuk-Jae Lee, Kiyoung Choi, and Jinho Lee. 2021. GradPIM: A practical processing-in-DRAM architecture for gradient descent. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 249–262.
- [38] Shine Kim, Yunho Jin, Gina Sohn, Jonghyun Bae, Tae Jun Ham, and Jae W Lee. 2021. Behemoth: a flash-centric training accelerator for extreme-scale {DNNs}. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 371–385.
- [39] Seung Soo Kim, Soo Kyeom Yong, Whayoung Kim, Sukin Kang, Hyeon Woo Park, Kyung Jean Yoon, Dong Sun Sheen, Seho Lee, and Cheol Seong Hwang. 2023. Review of semiconductor flash memory devices for material and process issues. *Advanced Materials* 35, 43 (2023), 2200659.
- [40] UK Kumar and BS Umashankar. 2007. Improved hamming code for error detection and correction. In *2007 2nd International Symposium on Wireless Pervasive Computing*. IEEE.
- [41] Yongkee Kwon, Kornjicuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoo Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeonju An, Jeongbin Kim, Jaewook Lee, Ilkon Kim, Jaehan Park, Chanwook Park, Yosub Song, Byeongsu Yang, Hyungdeok Lee, Seho Kim, Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeonpil Kang, Jungyeon Kim, Junyeol Jeon, Myeongjun Lee, Minyoung Shin, Minhwan Shin, Jaekyung Cha, Changson Jung, Kijoon Chang, Chunseok Jeong, Euicheol Lim, Il Park, Junhyun Chun, and Sk Hynix. 2022. System architecture and software stack for GDDR6-AiM. In *HCS*.
- [42] Jinho Lee, Jung Ho Ahn, and Kiyoung Choi. 2016. Buffered compares: Excavating the hidden parallelism inside DRAM architectures with lightweight logic. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1243–1248.
- [43] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyoungwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product. In *ISCA*.
- [44] Seungjae Lee, Chulbum Kim, Minsu Kim, Sung-min Joe, Joonsuc Jang, Seungbum Kim, Kangbin Lee, Jisu Kim, Jiyoung Park, Han-Jun Lee, et al. 2018. A 1Tb 4b/cell 64-stacked-WL 3D NAND flash memory with 12MB/s program throughput. In *2018 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 340–342.

- [45] Seungmin Lee, Joonsung Lim, Jun Hyoung Kim, Sunghwan Cho, Yong Kyu Lee, and Byoungdeog Choi. 2023. High-Density 3-D NAND Cell Array Design With Hybrid Bonding. *IEEE Transactions on Electron Devices* (2023).
- [46] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. 2020. DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator. *IEEE Computer Architecture Letters* 19, 2 (2020), 106–109.
- [47] Hamid Mahmoodi-Meimand and Kaushik Roy. 2004. Diode-footed domino: A leakage-tolerant high fan-in dynamic circuit design style. *IEEE Transactions on Circuits and Systems I: Regular Papers* 51, 3 (2004), 495–503.
- [48] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyang Feng, Simon Garcia De Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-mei Hwu. 2019. Deepstore: In-storage acceleration for intelligent queries. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 224–238.
- [49] Justin Meza, Qiang Wu, Sanjev Kumar, and Onur Mutlu. 2015. A large-scale study of flash memory failures in the field. *ACM SIGMETRICS Performance Evaluation Review* 43, 1 (2015), 177–190.
- [50] Oron Michael. 2016. NAND flash memory having internal ECC processing and method of operation thereof.
- [51] Rino Micheloni, Luca Crippa, and Alessia Marelli. 2010. *Inside NAND flash memories*. Springer Science & Business Media.
- [52] Micron. 2024. *Parallel NAND System Power Calculator*. <https://www.micron.com/support/tools-and-utilities/nand-system-power-calculator>
- [53] Neal Mielke, Todd Marquart, Ning Wu, Jeff Kessenich, Hanmant Belgal, Eric Schares, Falgun Trivedi, Evan Goodness, and Leland R Nevill. 2008. Bit error rate in NAND flash memories. In *2008 IEEE International Reliability Physics Symposium*. IEEE, 9–19.
- [54] Onur Mutlu. 2013. Memory scaling: A systems architecture perspective. In *2013 5th IEEE International Memory Workshop*. IEEE, 21–25.
- [55] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungrun. 2019. Enabling practical processing in and near memory for data-intensive computing. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–4.
- [56] Iyswarya Narayanan, Di Wang, Myeongjae Jeon, Bikash Sharma, Laura Caulfield, Anand Sivasubramaniam, Ben Cutler, Jie Liu, Badriddine Khessib, and Kushagra Vaid. 2016. SSD failures in datacenters: What? when? and why?. In *Proceedings of the 9th ACM International on Systems and Storage Conference*. 1–11.
- [57] V Shiva Prasad Nayak, Chinthaparthi Madhulika, and Uppugunduri Praval. 2017. Design of low power hamming code encoding, decoding and correcting circuits using reversible logic. In *2017 2nd IEEE international conference on recent trends in electronics, information & communication technology (RTEICT)*. IEEE, 778–781.
- [58] Open NAND Flash Interface (ONFI). 2022. *Open NAND Flash Interface Specification Revision 5.1*. <https://www.onfi.org/>.
- [59] Patrick E O'Neil, Elizabeth J O'Neil, and Xuedong Chen. 2007. The star schema benchmark (SSB). *Pat* 200, 0 (2007), 50.
- [60] Jisung Park, Roknoddin Azizi, Geraldo F Oliveira, Mohammad Sadrosadati, Rakesh Nadig, David Novo, Juan Gómez-Luna, Myungsuk Kim, and Onur Mutlu. 2022. Flash-Cosmos: In-Flash Bulk Bitwise Operations Using Inherent Computation Capability of NAND Flash Memory. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 937–955.
- [61] Jaehyun Park, Byeongho Kim, Sungmin Yun, Eojin Lee, Minsoo Rhu, and Jung Ho Ahn. 2021. Trim: Enhancing processor-memory interfaces with scalable tensor reduction in memory. In *MICRO*.
- [62] Greg Roelofs. 2024. *Portable Network Graphics (PNG)*. <http://www.libpng.org/pub/png/>
- [63] William Rurik and Arya Mazumdar. 2016. Hamming codes as error-reducing codes. In *2016 IEEE Information Theory Workshop (ITW)*. IEEE, 404–408.
- [64] Sun Il Shim, Jaehoon Jang, and Jaihyuk Song. 2023. Trends and future challenges of 3D NAND flash memory. In *2023 IEEE International Memory Workshop (IMW)*. IEEE, 1–4.
- [65] Siemens. 2024. *Calibre xRC*. <https://eda.sw.siemens.com/en-US/ic/calibre-design/circuit-verification/xrc/>
- [66] Panagiotis Sioulas, Periklis Chrysogelos, Manos Karpathiotakis, Raja Appuswamy, and Anastasia Ailamaki. 2019. Hardware-conscious hash-joins on gpus. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 698–709.
- [67] Synopsys. 2024. *HSPICE*. <https://www.synopsys.com/implementation-and-signoff/ams-simulation/primesim-hspice.html>
- [68] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. 2018. {MQSim}: A framework for enabling realistic studies of modern {Multi-Queue} {SSD} devices. In *16th USENIX Conference on File and Storage Technologies (FAST 18)*. 49–66.
- [69] Scott Temple, Walter Lau Neto, Ashton Snelgrove, Xifan Tang, and Pierre-Emmanuel Gaillardon. 2021. Getting the Most out of your Circuits with Heterogeneous Logic Synthesis. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1331–1334.
- [70] Ilya O Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. 2021. Mlp-mixer: An all-mlp architecture for vision. *Advances in neural information processing systems* 34 (2021), 24261–24272.
- [71] Inc Truevision. 1991. *Truevision Graphics Adapter (TGA)*. <https://www.dca.fee.unicamp.br/~martino/disciplinas/ea978/tgaffs.pdf>
- [72] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [73] Lei Wang and Naresh R Shanbhag. 2000. An energy-efficient noise-tolerant dynamic circuit technique. *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing* 47, 11 (2000), 1300–1306.
- [74] Xilinx. 2023. *Zynq ZCU102*. <https://www.xilinx.com/products/boards-and-kits/ek-u1-zcu102-g.html>
- [75] Neng-Fa Zhou and Håkan Kjellerstrand. 2016. The picat-sat compiler. In *Practical Aspects of Declarative Languages: 18th International Symposium, PADL 2016, St. Petersburg, FL, USA, January 18–19, 2016. Proceedings 18*. Springer, 48–62.
- [76] Chen Zou and Andrew A Chien. 2022. Assasin: Architecture support for stream computing to accelerate computational storage. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 354–368.